

N6_Visualitzacio_de_dades_comparacio_i_distribucions_pynb

November 26, 2025

0.1 Visualització de Comparacions

0.1.1 Gràfic de Barres

Aquest tipus de gràfic s'utilitza principalment quan es vol comparar un conjunt de valors i establir un rànquing clar entre ells.

Com que l'ull humà és molt bo comparant distàncies sobre un mateix eix, el gràfic de barres no només revela quins valors són més alts o més baixos, sinó que també ajuda a intuir amb precisió la diferència entre uns valors i uns altres.

A continuació mostrem un exemple de dades resultat de la representació de dades sobre el resum del total de partides presupostèries de la Generalitat dedicades a diferents àmbits. És un exemple fictici però es pots imaginar que és el resultat d'aplicar groupby a un munt de dades.

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt

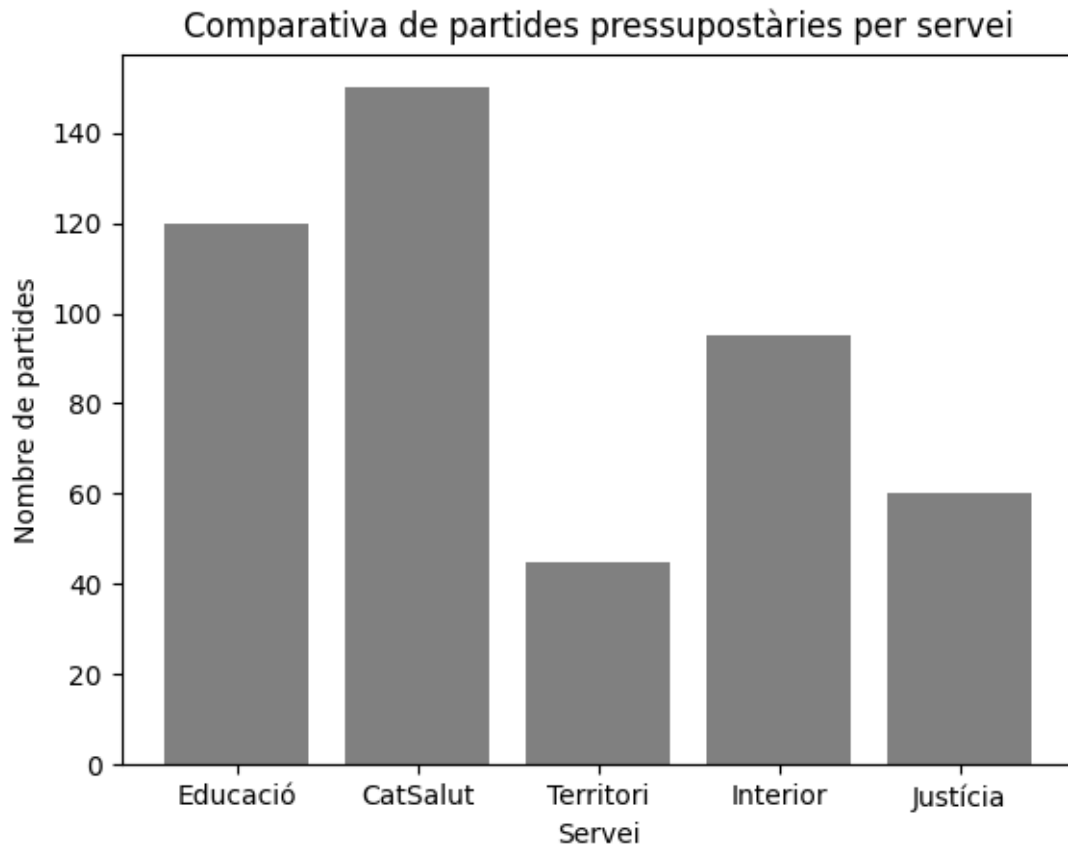
df = pd.DataFrame({
    'Servei': ['Educació', 'CatSalut', 'Territori', 'Interior', 'Justícia'],
    'Partides': [120, 150, 45, 95, 60]
})

df.head()
```

```
[ ]:      Servei  Partides
0  Educació      120
1  CatSalut      150
2  Territori      45
3  Interior      95
4  Justícia      60
```

Aquest tipus de dades categòriques per fer-ne una comparativa és un exemple clàssic d'ús de diagrames de barres.

```
[ ]: plt.bar(df['Servei'], df['Partides'], color='grey')
plt.xlabel('Servei')
plt.ylabel('Nombre de partides')
plt.title('Comparativa de partides pressupostàries per servei')
plt.show()
```



Podem intentar ajudar al nostre públic a entendre millor el missatge que volem donar o a poder interpretar millor els resultats:

- Ordenar les columnes ens ajudarà a comparar els valors.
- Adjuntar valor representatiu en la gràfica (mitjana, màxim història, etc.)
- Els colors també ens poden ajudar a destacar o que el públic posi el focus en alguna dada concreta.
- Adjuntar una llegenda si és necessari amb informació representativa.

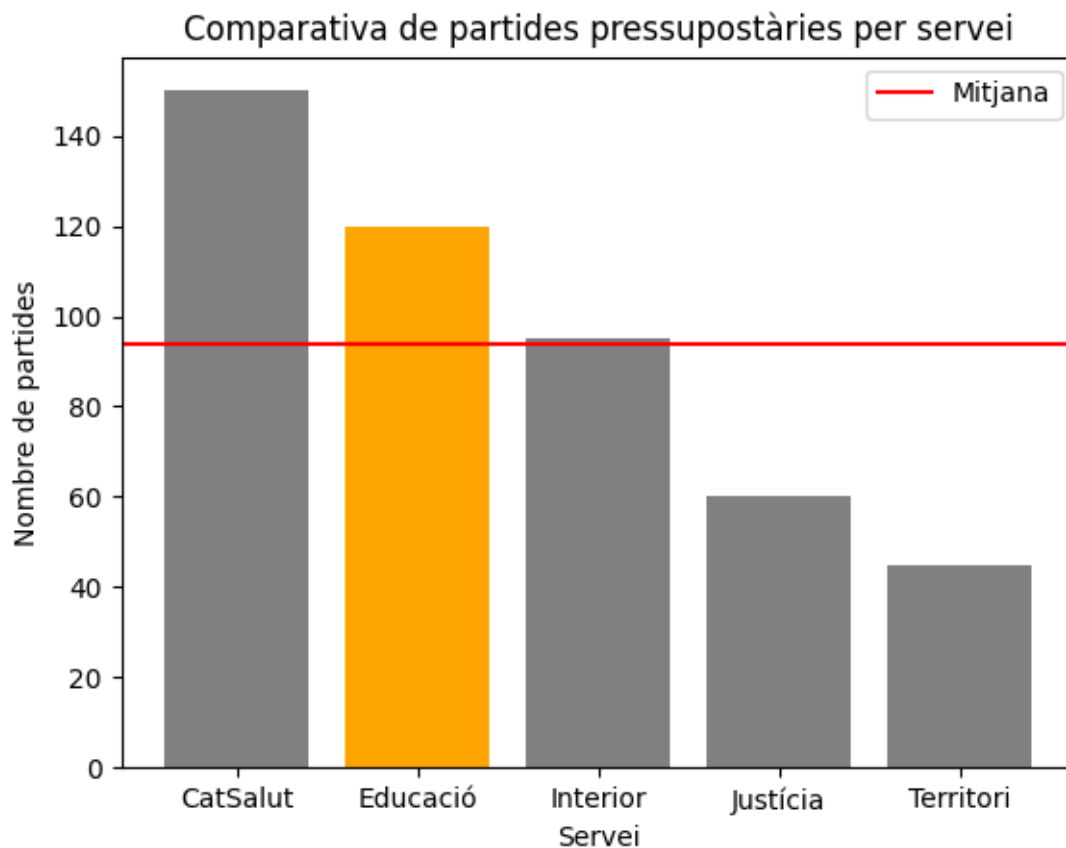
```
[ ]: # Ordenem les categories
df.sort_values('Partides', ascending=False, inplace=True)

#resaltem la barra d'educació
colors = ['gray' if servei != 'Educació' else 'orange' for servei in df['Servei']]

plt.bar(df['Servei'], df['Partides'], color=colors)
plt.xlabel('Servei')
plt.ylabel('Nombre de partides')
plt.title('Comparativa de partides pressupostàries per servei')
```

```
#afegir una línia amb el valor de la mitjana
plt.axhline(df['Partides'].mean(), color='red', linestyle='--', label='Mitjana')

#afegim una llegenda
plt.legend()
plt.show()
```



En alguns casos (moltes categories o elements amb noms molts llargs) els diagrames de barres horitzontals poden millorar-ne la llegibilitat.

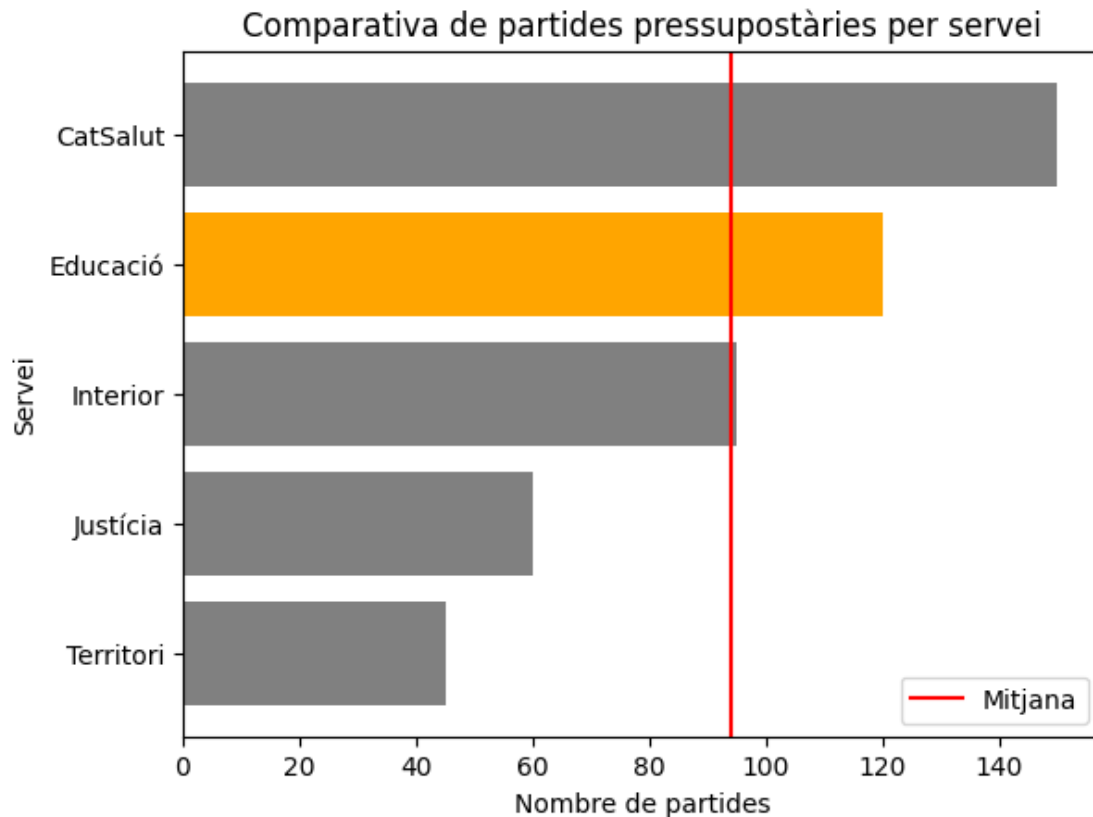
```
[ ]: # Utilitzem les mateixes opcions que en els cas anterior
df.sort_values('Partides', ascending=True, inplace=True)
colors = ['gray' if servei != 'Educació' else 'orange' for servei in df['Servei']]

# barh - gràfic de barres HORIZONTAL
plt.barh(df['Servei'], df['Partides'], color=colors)

plt.ylabel('Servei')
```

```
plt.xlabel('Nombre de partides')
plt.title('Comparativa de partides pressupostàries per servei')
plt.axvline(df['Partides'].mean(), color='red', linestyle='-', label='Mitjana')
plt.legend()

plt.show()
```



Tot i així, quan es representen dades temporals (i les categories són hores, dies, mesos, etc.) cal ordenar l'eix de manera temporal i es recomana la disposició vertical de les barres.

0.1.2 Barres Agrupades

És una versió del gràfic de barres anterior però en aquest cas hem ajuntat diversos gràfics de barres sobre un mateix eix.

Aquest tipus de gràfic serveix per descompondre una categoria en diferents segments i poder veure la distribució dels valors dins d'aquesta categoria.

És més fàcil d'utilitzar la directament la llibreria seaborn per aquest tipus de gràfics (en matplotlib s'ha de fer més "manual").

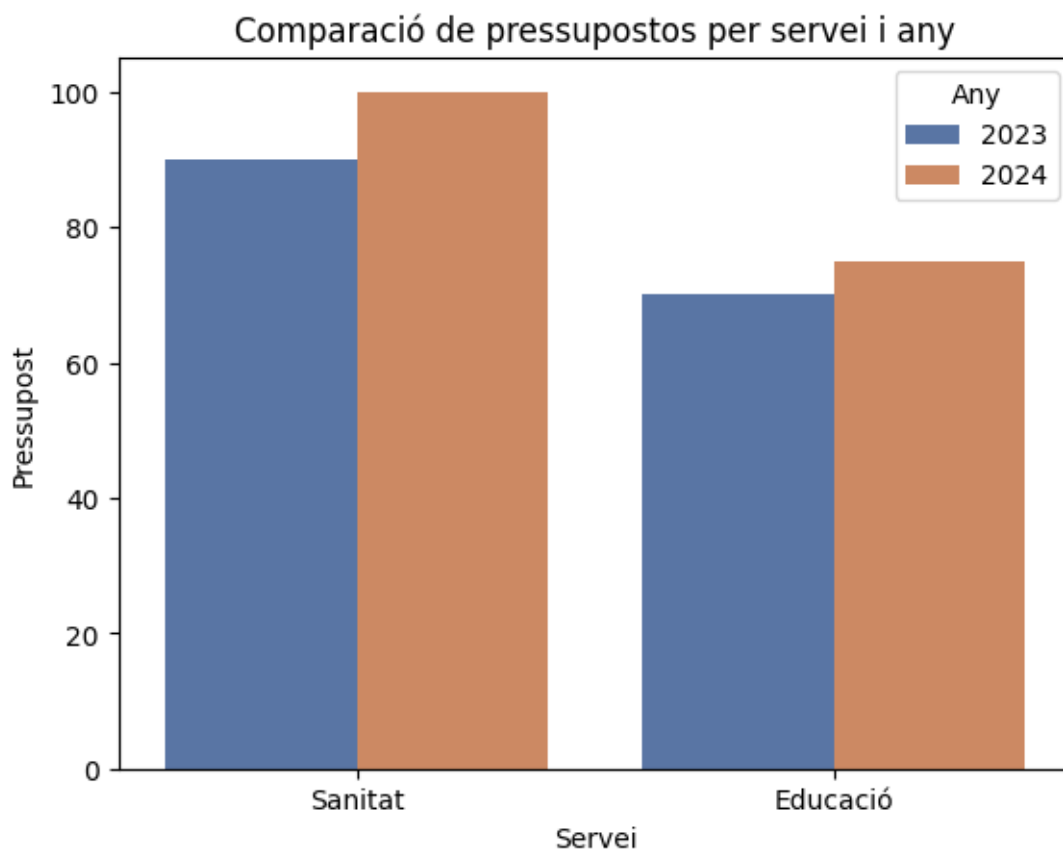
En aquest cas, utilitzem les mateixes dades que en el cas anterior però mostrant la seva evolució d'un any al següent:

```
[ ]: import seaborn as sns
```

```
df = pd.DataFrame({  
    'Any': [2023, 2023, 2024, 2024],  
    'Servei': ['Sanitat', 'Educació', 'Sanitat', 'Educació'],  
    'Pressupost': [90, 70, 100, 75]  
})  
  
df.head()
```

```
[ ]:      Any    Servei  Pressupost  
0  2023    Sanitat         90  
1  2023  Educació         70  
2  2024    Sanitat        100  
3  2024  Educació         75
```

```
[ ]: sns.barplot(data=df, x='Servei', y='Pressupost', hue='Any', palette = 'deep')  
#altres 'palette' disponibles : deep (per defecte), pastel, muted, bright, viridis, etc.  
  
plt.title('Comparació de pressupostos per servei i any')  
plt.show()
```

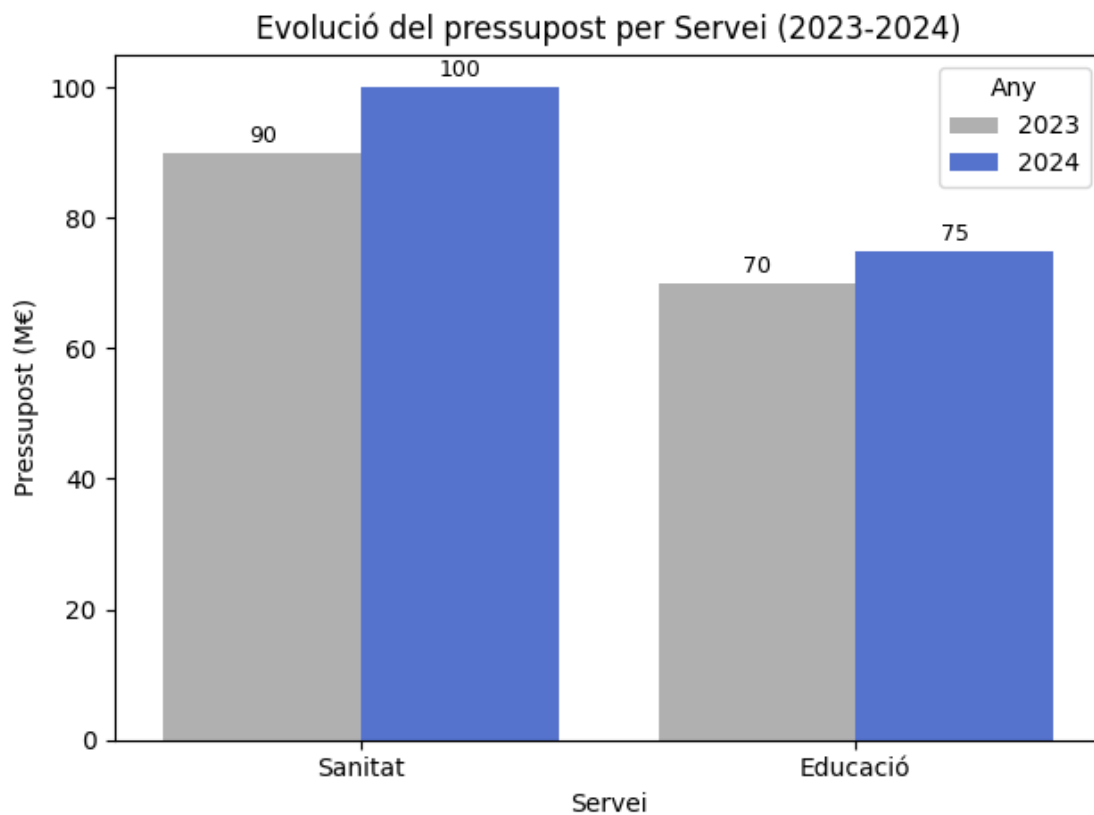


Veiem algunes possibles millores que podem aplicar: * Podem definir una paleta *custom* pq ens permeti diferenciar segons any, pq ens interessa intensificar el missatge que els pressupostos han augmentat. * Afegir dades més concretes als eixos (períodes, unitats).

```
[ ]: paleta_custom = ['#b0b0b0', '#4169E1'] # gris clar i blau
ax = sns.barplot(data=df, x='Servei', y='Pressupost', hue='Any',
                 palette=paleta_custom)

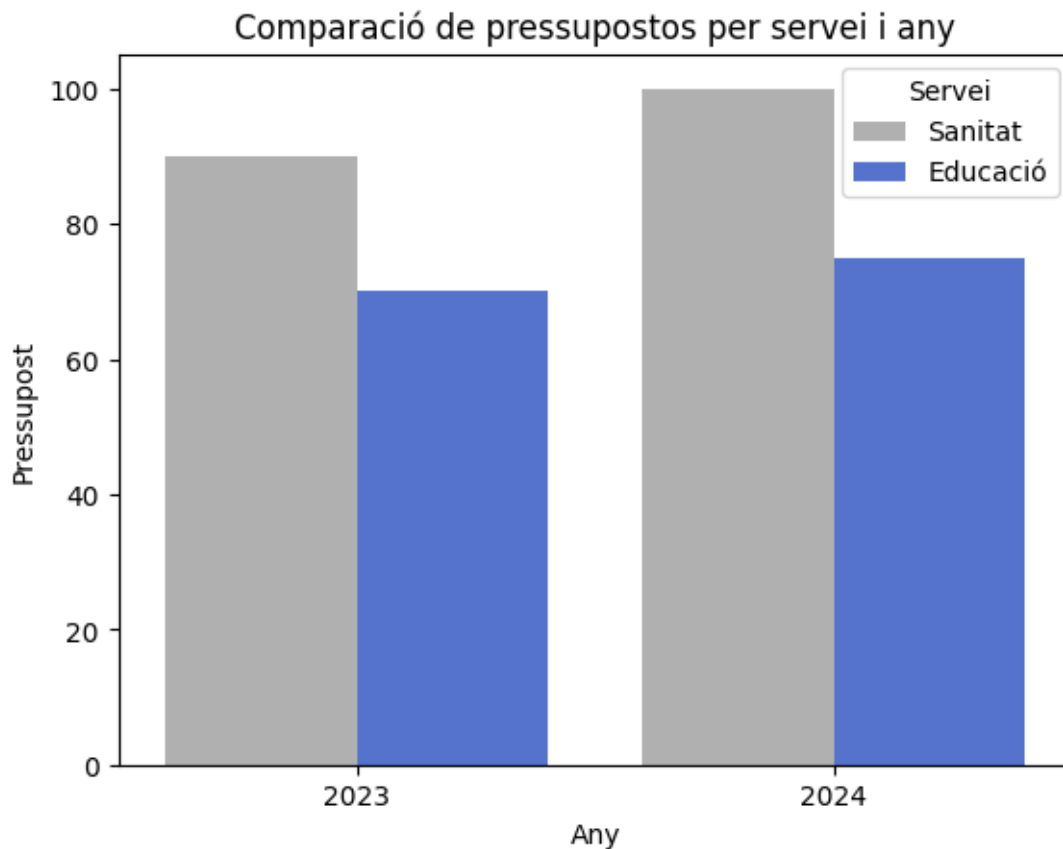
# Afegir els valors sobre cada barra: s'ha de fer de manera manual i accedint a
# les propietats dels eixos de la gràfica.
for c in ax.containers:
    ax.bar_label(c, padding=2, fontsize=9)

plt.title('Evolució del pressupost per Servei (2023-2024)')
plt.ylabel('Pressupost (M€)')
plt.xlabel('Servei')
plt.tight_layout()
plt.show()
```



```
[ ]: #paleta_custom = paleta_custom[::-1]
sns.barplot(data=df, x='Any', y='Pressupost', hue='Servei', palette =
    ↪paleta_custom)
#altres 'palette' disponibles : deep (per defecte), pastel, muted, bright,
    ↪viridis, etc.

plt.title('Comparació de pressupostos per servei i any')
plt.show()
```



Quin creus que és el missatge o la intenció del gràfic en cada cas?

El primer gràfic fa la comparació per servei per any, mentre que el segon concentra la comparació per any segons el servei. La tria depèn de si volem prioritzar la comparació de serveis al llarg dels anys o la comparació de serveis dins de cada any.

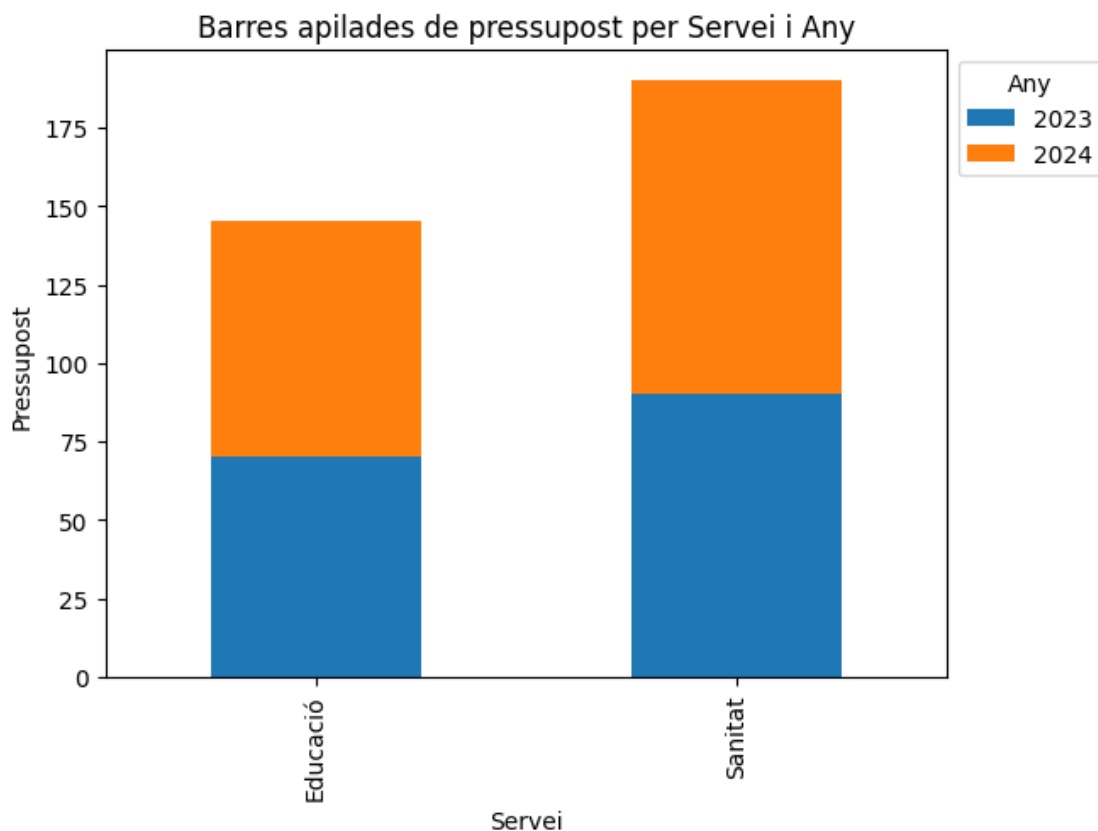
En aquest tipus de gràfics és important no sobrecarregar-lo amb masses categories (n'acabarien dificultant la comprensió) i donat el cas, és recomanable dividir el gràfic en diferents gràfics de barres separats.

0.1.3 Barres Apilades

El gràfic de barres apilades serveix per veure el total d'un conjunt i com es reparteix entre diferents parts (segments). Cada barra mostra la suma total i els diferents segments indiquen la proporció que aporta cada element.

Aquest tipus de gràfic pot ser poc clar si es vol comparar la mida dels segments entre barres diferents. Per això, s'utilitza sobretot quan cada barra representa el 100% d'un total i volem mostrar els percentatges de cada part dins d'aquest conjunt. En aquests casos és més útil i entenedor que el gràfic de “formatge”.

```
[ ]: df.groupby(by=['Servei', 'Any'])['Pressupost'].sum().unstack().plot(kind='bar',  
    stacked=True)  
plt.legend(title='Any', bbox_to_anchor=(1.0, 1), loc='upper left')  
plt.title('Barres apilades de pressupost per Servei i Any')  
plt.ylabel('Pressupost')  
plt.xlabel('Servei')  
plt.show()
```



Amb Seaborn Object aquest tipus de gràfics serà més fàcil.

0.2 Noves tendències

0.2.1 Seaborn API

La nova API orientada a objectes de Seaborn és una tendència recent en la visualització de dades amb Python que ofereix un enfocament més modular i flexible comparat amb la forma clàssica basada en funcions com `sns.barplot()`.

A diferència de l'API tradicional, aquesta nova forma de treballar crea objectes gràfics que es poden modificar, combinar i encadenar de manera més natural.

Utilitza la classe `so.Plot` que defineix el conjunt de dades i les variables, després afegeix capes visuals (com barres, línies o punts) amb `.add()`.

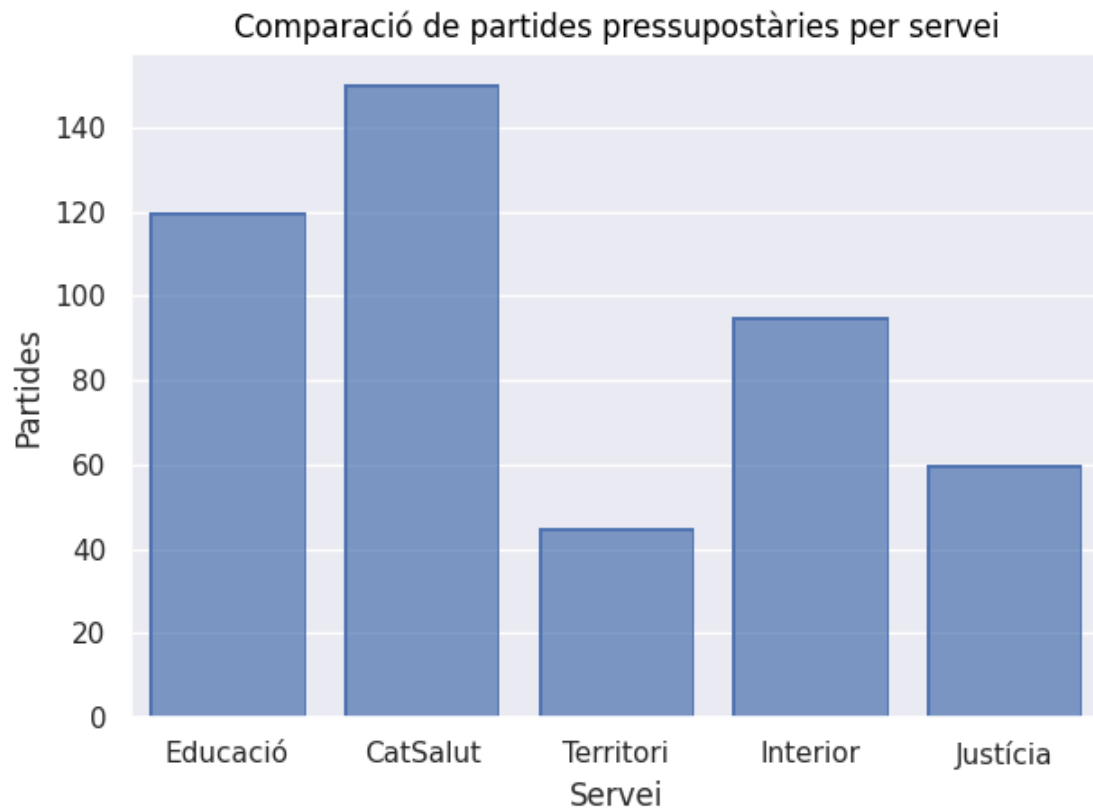
```
[ ]: import seaborn.objects as so
```

0.2.2 Barres simples:

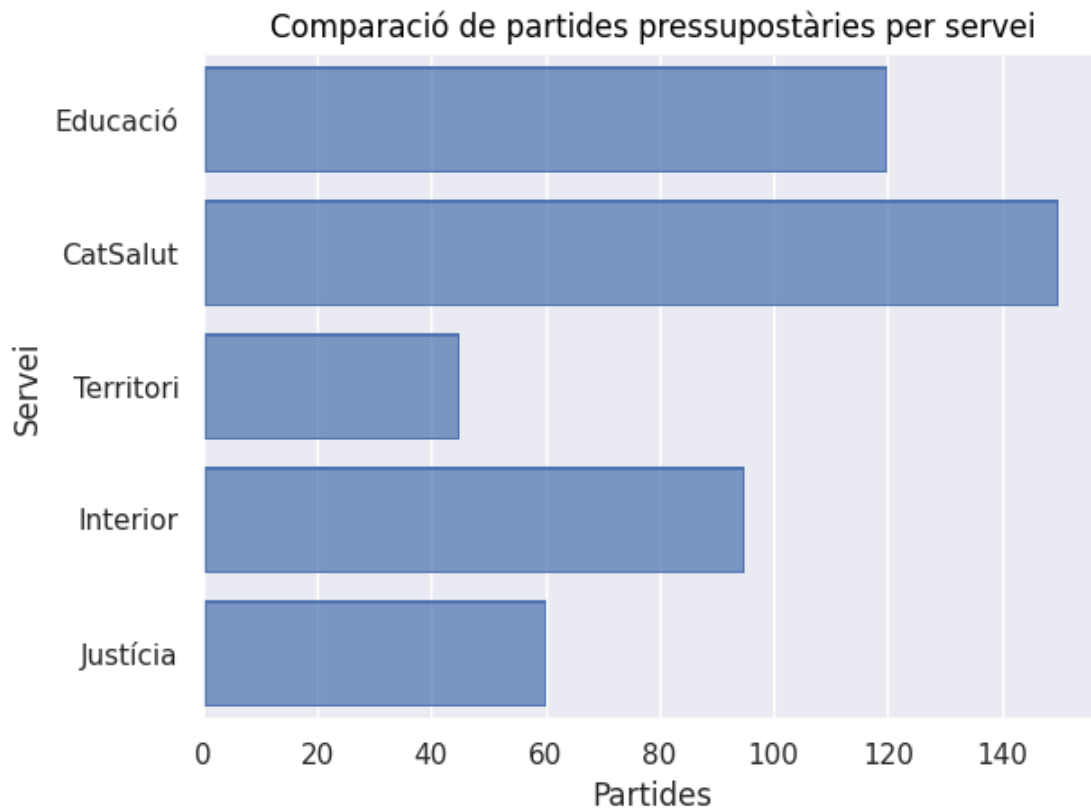
```
[ ]: df = pd.DataFrame({
    'Servei': ['Educació', 'CatSalut', 'Territori', 'Interior', 'Justícia'],
    'Partides': [120, 150, 45, 95, 60]
})

p = (
    #so.Plot(df, x="Servei", y="Partides", color="Servei")
    so.Plot(df, "Servei", "Partides")
    .add(so.Bar())
    .label(title="Comparació de partides pressupostàries per servei")
)

p.show()
```



```
[ ]: p = (  
    so.Plot(df, "Partides", "Servei")  
    .add(so.Bar())  
    .label(title="Comparació de partides pressupostàries per servei")  
)  
  
p.show()
```

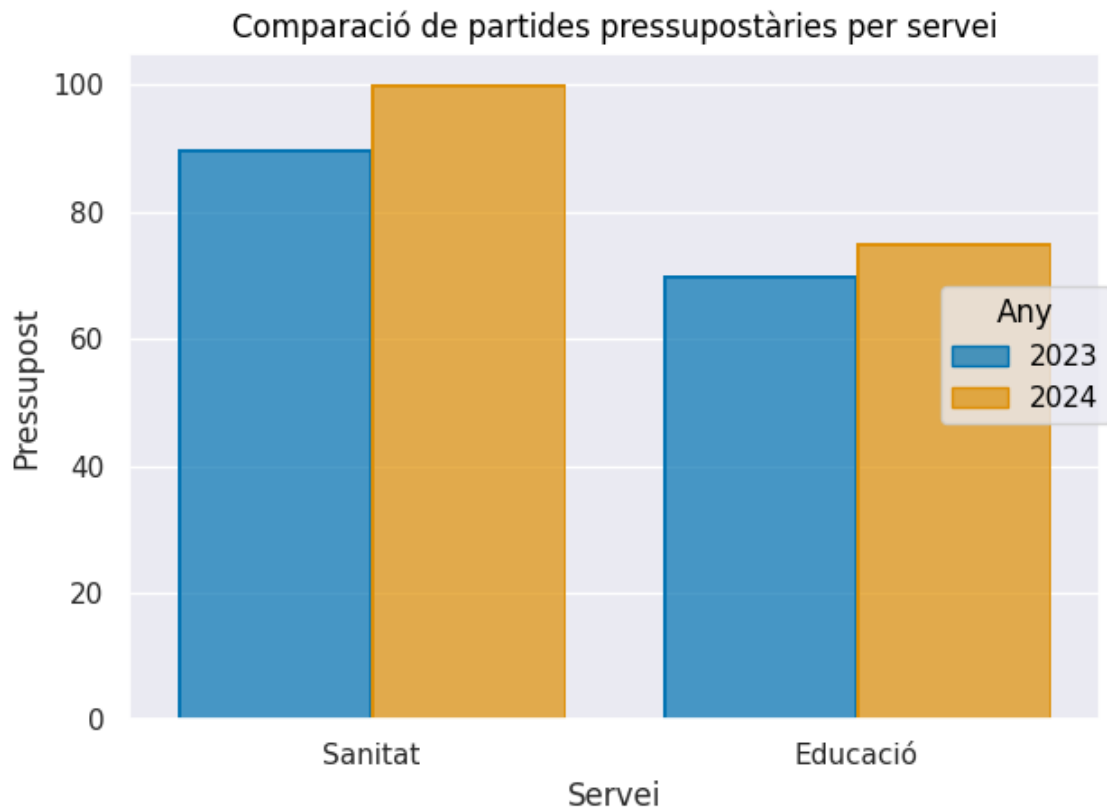


Barres agrupades:

```
[ ]: df = pd.DataFrame({
    'Any': [2023, 2023, 2024, 2024],
    'Servei': ['Sanitat', 'Educació', 'Sanitat', 'Educació'],
    'Pressupost': [90, 70, 100, 75]
})

p = (
    so.Plot(df, x="Servei", y="Pressupost", color="Any")
    .add(so.Bar(), so.Dodge())
    .scale(color="colorblind") #dark, pastel, etc
    .label(title="Comparació de partides pressupostàries per servei")
)

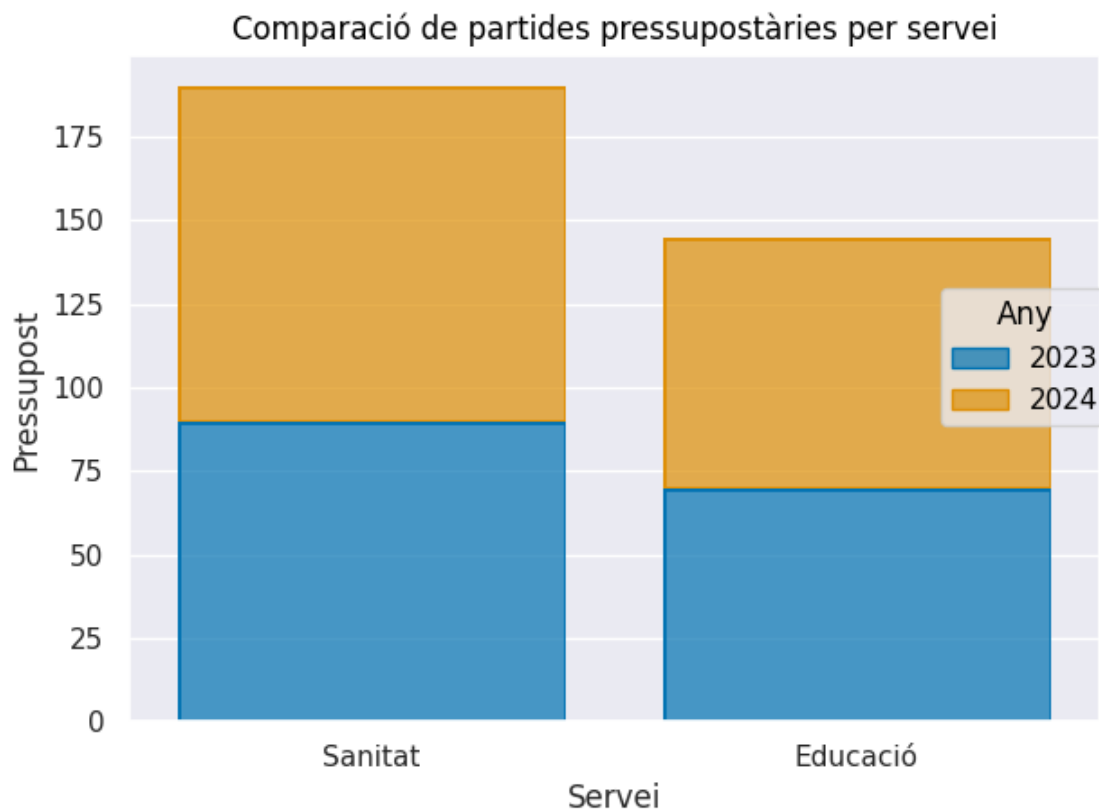
p.show()
```



0.2.3 Barres apilades:

```
[ ]: p = (
    so.Plot(df, x="Servei", y="Pressupost", color="Any")
    .add(so.Bar(), so.Stack())
    .scale(color="colorblind")
    .label(title="Comparació de partides pressupostàries per servei")
)

p.show()
```



És una eina potent però encara no té totes les opcions que tenim amb matplotlib però podem combinar-les.

0.3 Visualització de distribucions

0.3.1 Histograma

Representació gràfica basada en barres que mostra la distribució de freqüències d'una variable quantitativa contínua (o discreta si té molts valors).

El seu objectiu principal és entendre com es reparteixen els valors d'una variable dins de tot el conjunt de dades, permetent visualitzar si tendeixen a agrupar-se al voltant de la mitjana (distribució normal, típica en forma de campana) o si s'acumulen cap a un extrem (distribució asimètrica).

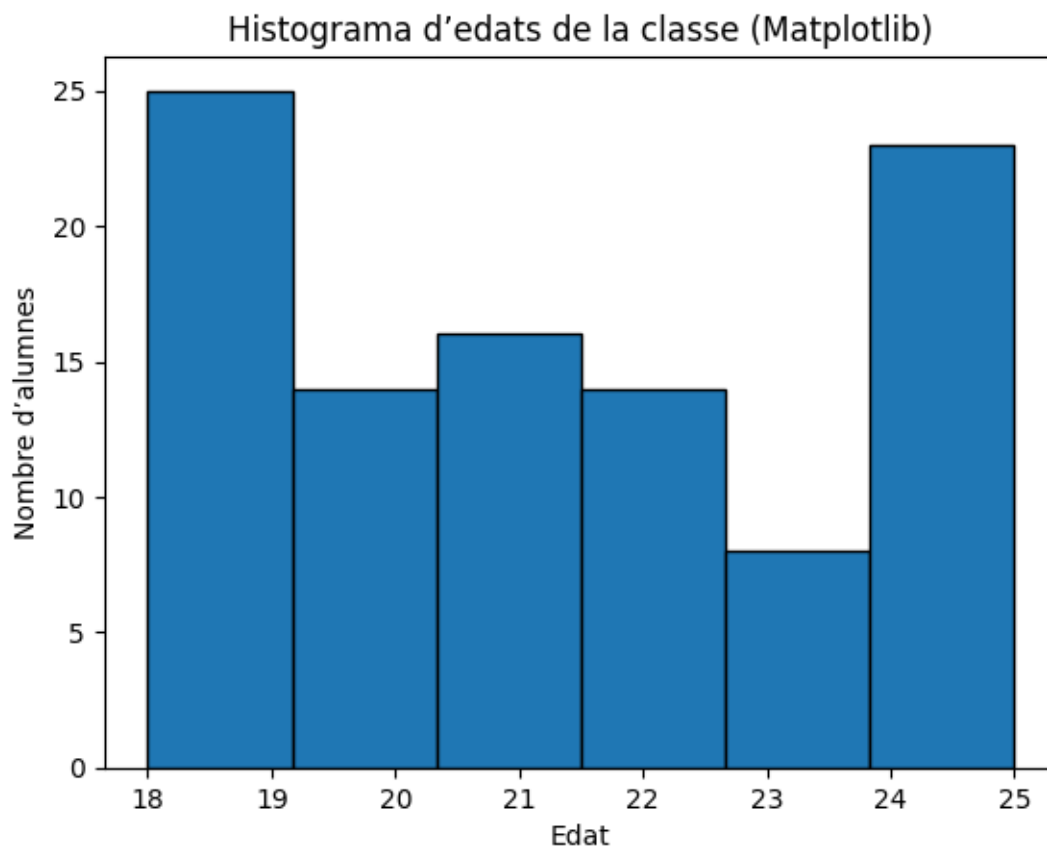
```
[ ]: import numpy as np

edats = np.random.randint(18, 26, size=100)
df_edats = pd.DataFrame({'Edat': edats})

df_edats.head()
```

```
[ ]:  Edat
      0    23
      1    22
      2    25
      3    22
      4    19
```

```
[ ]: plt.hist(df_edats['Edat'], bins=6, edgecolor='black')
      plt.title('Histograma d'edats de la classe (Matplotlib)')
      plt.xlabel('Edat')
      plt.ylabel('Nombre d'alumnes')
      plt.show()
```



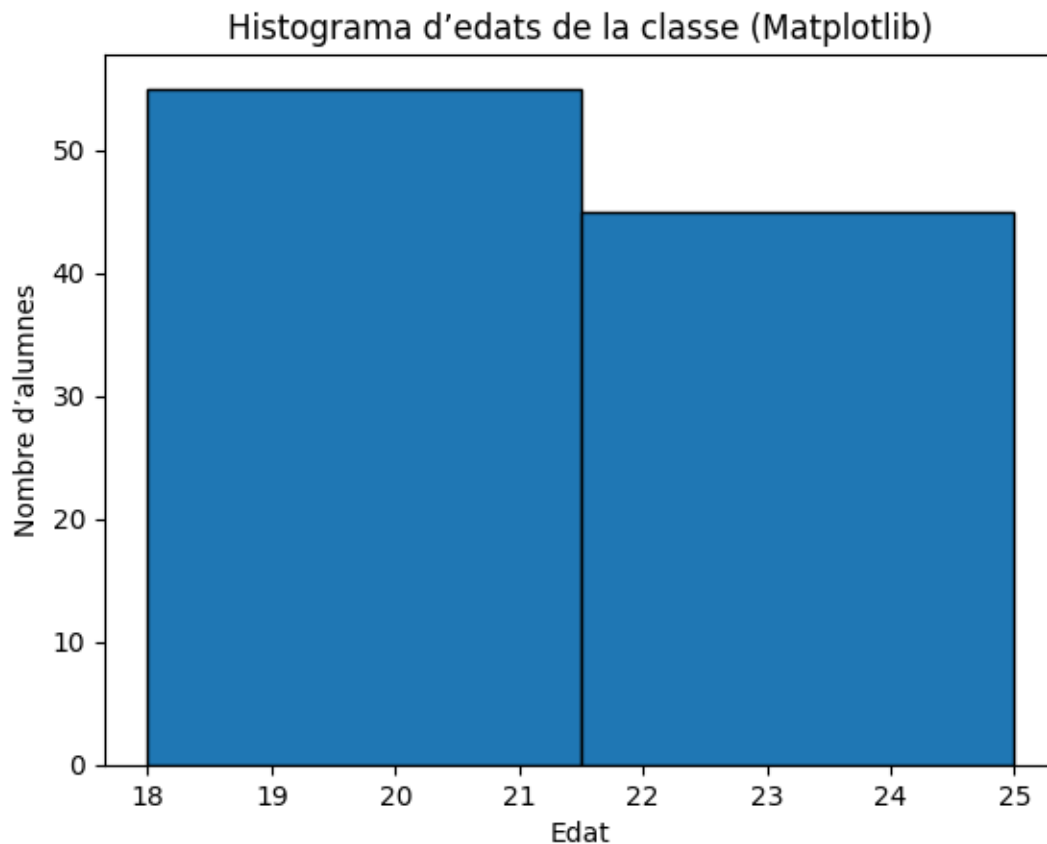
El nombre de barres (o bins) en un histograma determina la mida dels intervals en què s'agrupen els valors de dades. Generalment, els programes seleccionen automàticament la mida dels intervals en funció de la distribució i rang dels valors presents, però aquesta mida es pot ajustar manualment per obtenir una visió més precisa o diferent de la distribució.

No existeix una fórmula exacta per triar la mida ideal dels intervals, ja que aquest depèn del conjunt de dades i de l'objectiu de l'anàlisi.

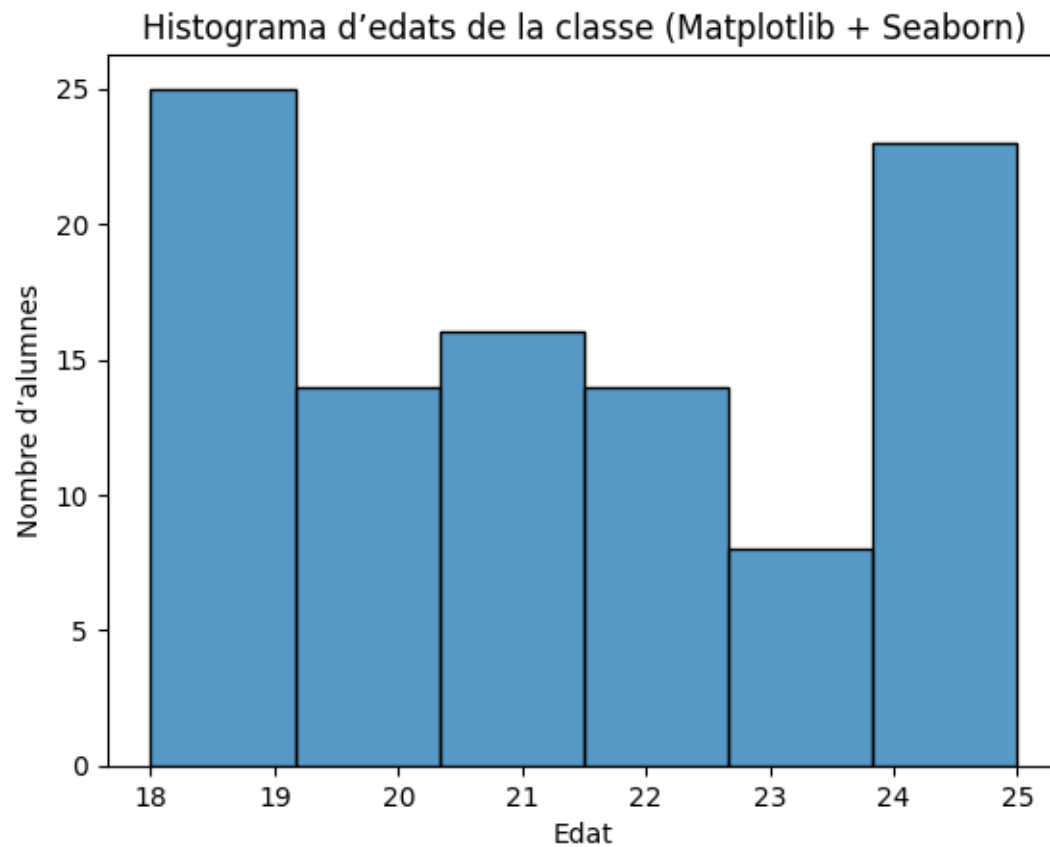
Alguns intervals massa amples poden amagar detalls importants i donar una visió massa generalitzada, mentre que intervals molt petits poden generar un histograma molt fragmentat i difícil d'interpretar.

És recomanable que l'analista de dades provi diversos valors de la mida dels intervals per veure com es modifica la forma de l'histograma i extraure conclusions més acurades.

```
[ ]: plt.hist(df_edats['Edat'], bins=2, edgecolor='black')
plt.title('Histograma d'edats de la classe (Matplotlib)')
plt.xlabel('Edat')
plt.ylabel('Nombre d'alumnes')
plt.show()
```



```
[ ]: sns.histplot(df_edats['Edat'], bins=6)
plt.title('Histograma d'edats de la classe (Matplotlib + Seaborn)')
plt.xlabel('Edat')
plt.ylabel('Nombre d'alumnes')
plt.show()
```



```
[ ]: p = (  
    so.Plot(df_edats, x="Edat")  
    .add(so.Bar(), so.Hist(bins=3))  
    .label(title="Histograma")  
)  
p.show()
```